LAPORAN PRAKTIKUM

STRUKTUR DATA

Stack

disusun Oleh:

MUHAMAD FAJRI AULIA

NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 9 APRIL 2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul "**Stack**". Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Stack dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 9 April 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM 1
KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Persyaratan Praktikum 1
1.4 Waktu dan tempat pelaksanaan 2
BAB II PEMBAHASAN 3
2.1 Pengertian Stack 3
2.2 Operasi dasar pada sebuah stack 3
2.3 Langkah Pengerjaan 3
2.3.1 Nilai Maksimum 3
2.3.2 Siswa Stack 4
2.3.3 Stack Array 7
2.3.4 Stack Array Driver 8
2.3.5 Stack Postfix 9
BAB III KESIMPULAN DAN SARAN 11
3.1 Kesimpulan 11
3.2 Saran 11
DAFTAR PUSTAKA 12

DAFTAR GAMBAR

Gambar 1 4
Gambar 2 6
Gambar 3 9
Gambar 4 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Stack atau tumpukan merupakan sebuah struktur data linier yang mengikuti urutan tertentu Dimana operasi dilakukan. Biasanya urutannya LIFO (Last In First Out) yang mana ini maksudnya bahwa elemen yang dimasukkan terakhir akan keluar pertama dan menyatakan bahwa element yang dimasukkan pertama akan keluar terakhir.

1.2 Tujuan Praktikum

- Memahami konsep dasar Stack di Java.
- Mengetahui cara membuat dan menggunakan Stack
- Melatih penggunaan method yang ada pada Stack.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan tempat pelaksanaan

Pelaksanakan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanankan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Stack (tumpukan) adalah struktur data linier yang mengikuti urutan tertentu di mana operasi dilakukan. Pada stack ada beberapa jenis diantaranya stack ukuran tetap yang mana ini stack ukurannya tidak dapat bertambah. Jika stack penuh dan ada Upaya untuk menambahkan elemen kedalamnya maka akan terjadi kesalahan overflow jika stack kosong maka aka nada Upaya untuk menghapus elemen darinya maka akan terjadi kesalahan underflow.

Stack ukuran dinamis ini merupakan sebuah stack yang dapat bertambah atau berkurang secara dinamis yang mana jika stack penuh maka secara otomatis meningkatkan ukurannya untuk mengakomodasi elemen baru dan Ketika stack kosong stack mengurangi ukurannya. Jenis ini diimplementasikan menggunakan linked list karena memungkinkan untuk mengubah ukuran stack dengan mudah.

2.2 Operasi dasar pada sebuah stack

Beberapa operdasi dasar dari sebuah stack adalah sebagai berikut :

- Push : Menambahkan elemen baru pada tumpukan.
- Pop : Menghapus dan mengembalikan elemen teratas dari tumpukan.
- Peek : Mengembalikan elemen teratas pada stack.
- isEmpty : Mememriksa apakah tumpukan kosong.
- Size : Menemukan jumlah elemen dalam tumpukan.

2.3 Langkah Pengerjaan

Untuk lebih memahami kita akan mulai membahas program berikut.

2.3.1 Nilai Maksimum

Untuk kode programnya sebagai berikut.

```
package pekan3_2511532009;
import java.util.*;

public class NilaiMaksimum_2511532009 {
	public static int max_2009 (Stack<Integer> s_2009) {
		Stack<Integer> backup = new Stack<Integer>();
		int maxValue_2009 = s_2009.pop ();
		backup.push(maxValue_2009);
		while (!s_2009.isEmpty() ) {
```

```
                int next = s_2009.pop ();
                backup.push(next);
                maxValue_2009 = Math.max(maxValue_2009, next);
            }
            while (!backup.isEmpty()) {
                s_2009.push (backup.pop());
            }
            return maxValue_2009;
        }
        public static void main(String[] args) {
            Stack<Integer> s_2009 = new Stack<Integer> ();
            s_2009.push(70);
            s_2009.push(12);
            s_2009.push(20);
            System.out.println("isi stack " + s_2009);
            System.out.println("Stack Teratas " + s_2009.peek ());
            System.out.println("Nilai maksimum " + max_2009(s_2009));


        }

}
```

Pada program ini kita membuat sebuah contoh stack sederhana yang mana pada program ini kita menyatakan stack nya yang bertipe integer dengan nama backup. Jadi perintah perintah operasi dasar kita menggunakan backup.operasi. Pada program kita melakukan operasi push pada maxvalue atau s. Lalu dibuat While dengan keadaan memeriksa s isEmpty maka akan dilakukan operasi pop dan push. Lalu di lanjutkan dengan while stack !backup is empty maka aka nada perintah push pada s dan pop pad backup. Lalu kembalikan nilai maxvalue. Lalu pada bagian akhir dilakukan cetakan hasil dari program yang mana pada akhir program kita langsung memasukkan nilai dari s dengan push. Dan di cetak hasilnya sehingga nilai yang dihasilkan adalah sebagai berikut.

```
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 1

2.3.2 Siswa Stack

Untuk program dari Daftar Kata ada sebagai berikut.

```
package pekan3_2511532009;
import java.util.*;
```

```java
class Siswa_2009 {
	String nama_2009;
	int nim_2009;

	public Siswa_2009(String nama_2009, int nim_2009) {
		this.nama_2009 = nama_2009;
		this.nim_2009 = nim_2009;
	}

	@Override
	public String toString() {
		return "Nim: " + nim_2009 + ", Nama: " + nama_2009;
	}
}

public class SiswaStack_2511532009 {
	private ArrayList<Siswa_2009> stack_2009;

	public SiswaStack_2511532009() {
		stack_2009 = new ArrayList<>();
	}

	public void push_2009(Siswa_2009 mhs) {
		stack_2009.add(mhs);
	}
	public Siswa_2009 pop () {
		if (!isEmpty() ) {
			return stack_2009.remove(stack_2009.size() - 1);
		}
		return null;
	}
	public Siswa_2009 peek() {
		if (!isEmpty()) {
			return stack_2009.get(stack_2009.size() - 1);
		}
		return null;
	}

	public boolean isEmpty() {
		return stack_2009.isEmpty();
	}

	public void tampilkanSiswa_2009() {
		for (int i = stack_2009.size() - 1; i >= 0; i--) {
			System.out.println(stack_2009.get(i));
		}
	}
	public static void main (String[] args) {
		SiswaStack_2511532009 studentStack_2009 = new
SiswaStack_2511532009();

		Siswa_2009 mhs1_2009 = new Siswa_2009("Ali", 1);
		Siswa_2009 mhs2_2009 = new Siswa_2009("Boby", 2);
		Siswa_2009 mhs3_2009 = new Siswa_2009("Charles", 3);
		studentStack_2009.push_2009(mhs1_2009);
		studentStack_2009.push_2009(mhs2_2009);
		studentStack_2009.push_2009(mhs3_2009);

		System.out.println("Siswa di dalam staack:");
```

```
            studentStack_2009.tampilkanSiswa_2009();

            System.out.println("siswa teratas " + studentStack_2009.peek());
            studentStack_2009.tampilkanSiswa_2009();

            System.out.println("siswa teratas " + studentStack_2009.peek());
            System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack_2009.pop());
            System.out.println("daftar siswa setelah di pop : ");
            studentStack_2009.tampilkanSiswa_2009();
        }

}
```

Pada program ini kita membuat sebuah program yang mengimplementasikan stack menggunakan ArrayList yang mana disini ada 2 kelas utama yaitu kelas siswa_2009 dan siswastack_2511532009. Langkah awal kita membuat sebuah konstruktor untuk menginisiasi nilai dari nama dan nim. Lalu pada bagian kelas siswastack kita gunakan untuk membuat atau mengelola data dalam bentuk stack kelas ini menggunakan array list sebagai media penyimpanan data. Selanjutnya kita menggunakan method add , pop, peek, dan isEmpty. Lalu pada bagian akhir program menampilkan seluruh isi stack. Dengan output yang dihasilkan adalah sebagai berikut.

```
<terminated> SiswaStack_2511532009 [Java Application] C:\Users\LoQ\.p2\pool\plugins
Siswa di dalam staack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Gambar 2

2.3.3 Stack Array

Untuk program dari Stack Array adalah sebagai berikut.

```
package pekan3_2511532009;
import java.util.*;

public class stackArray_2511532009 {
            static final int MAX_2009 = 1000;
            int top_2009;
            int a[] = new int [MAX_2009];
            boolean isEmpty_2009()
            {
                  return (top_2009 < 0);
            }
            stackArray_2511532009()
            {
                  top_2009 = -1;
            }
            boolean push_2511532009 (int x)
            {
                  if (top_2009 >= (MAX_2009 - 1)) {
                        System.out.println("Stack Overflow");
                        return false;
                  }
                  else {
                        a[++top_2009] = x;
                        System.out.println(x + " dimasukkan dalam stack");
                        return true;
                  }
            }
            int pop_2511532009()
            {
                  if (top_2009 < 0) {
                        System.out.println("Stack Underflow");
                        return 0;
                  }
                  else {
                        int x = a [top_2009--];
                        return x;
                  }
}
            int peek_2511532009 ()
            {
                  if (top_2009 < 0) {
                        System.out.println("Stack Underflow");
                        return 0;
                  }
                  else {
                        int x = a[top_2009];
                        return x;
                  }
            }
            void print_2511532009 () {
                  for(int i = top_2009;i>-1;i--) {
                        System.out.println(" " + a[i]);
                  }
            }

}
```

Pada program ini kita mengguunakan konstanta dengan kapasitas maksimum 1000 element. Selanjutnya kita buat konstruktornya yang mana menyatakan bahwa stack itu dalam keadaan kosong atau -1 lalu kita membuat sebuah fungsi pengecekkan apakah stack itu kosong atau tidak. Lalu kita membuat fungsi push , pop, peek dan print untuk melihat hasil dari program. Disini kita hanya membuat programnya yang mana untuk melihat hasilnya kita menggunakan atau membuat drivernya.

2.3.4 Stack Array Driver

Untuk program dari Mahasiswa adalah sebagai berikut.

Oke pada program ini kita menyatakan tipe dari sebuah variabel yang mana pada program in ada nim, nama, prodi nah lalu kita buat sebuah getternya lalu kita buat overridenya. Untuk programnya belum bisa di run karena bisa Ketika sudah buat drivernya.

```
package pekan3_2511532009;

public class stackArrayDriver_2511532009 {
	public static void main (String[] args ) {
		stackArray_2511532009 s = new stackArray_2511532009();
		s.push_2511532009 (10);
		s.push_2511532009 (20);
		s.push_2511532009 (30);
		System.out.println(s.pop_2511532009() + " dikeluarkan dari stack");
		System.out.println("Elemen teratas adalah :" + s.peek_2511532009());
		System.out.println("Element pada stack : ");
		s.print_2511532009();
	}

}
```

Pada program ini yang mana ini merupakan driver dari stackarray tadi disini kita memasukkan data data nya menggunakan push lalu kita melakukan pop peek dan hasilnya bisa dilihat. Dengan output yang dihasilkan adalah sebagai berikut.

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack :
 20
 10
```

Gambar 3

2.3.5 Stack Postfix

Pada program mahasiswa driver adalah proses penjalanan dari program tadi. Dengan program adalah sebagai berikut.

```java
package pekan3_2511532009;
import java.util.*;
public class StackPostfix_2511532009 {
	public static int postfixEvaluate_2511532009(String expression) {
		Stack<Integer> s_2511532009 = new Stack<Integer>();
		Scanner input = new Scanner(expression);
		while (input.hasNext()) {
			if (input.hasNextInt()) { // an operand (integer)
				s_2511532009.push(input.nextInt());
			} else {
				String operator = input.next();
				int operand2 = s_2511532009.pop();
				int operand1 = s_2511532009.pop();
				if (operator.equals("+")) {
					s_2511532009.push(operand1 + operand2);
				} else if (operator.equals("-")) {
					s_2511532009.push(operand1 - operand2);
				} else if (operator.equals("*")) {
					s_2511532009.push(operand1 * operand2);
				} else {
					s_2511532009.push(operand1 / operand2);
				}
			}
		}
		input.close();
		return s_2511532009.pop();
	}
	public static void main (String[] args) {
		System.out.println("Hasil postfix= " + postfixEvaluate_2511532009("5
2 4 * + 7 -"));
	}


}
```

Pada program ini kita membuat sebuah stack yang mana menyimpan data sederhana lalu akan dilakukan yang namanya proses perhitungan. Di awal kita mulai dengan inisialisasi stack yang mana stack bertipe integer untuk menyimpan angka. Lalu kita buat sebuah operand yang mana akan melakukan operasi jika ada data yang mana tiap pilihan akan menentukan apa perintah yang akan dilakukan. Hasil operasi akan dimunculkan pada hasil akhir dan akan di ambil menggunakan pop. Yang mana pada program ini akan menampilkan hasil dari operasi yang sudah dibuat sehingga output yang diberikan adalah sebagai berikut.

```
<terminated> StackPostfix_251153
Hasil postfix= 6
```

Gambar 4

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Stack adalah sebuah struktur data yang diimplementasikan menggunakan array list dan stack pada java. Stack seperti penjelasannya menggunakan konsep LIFO atau Last In First Out yang mana ini artinya data terakhir masuk adalah data pertama yang keluar. Dari praktikum ini diharapkan dapat memahami konsep dari penggunaan stack secara sederhana

3.2 Saran

- Ulangi dan terus Latihan agar makin memahami konsep dari Stack.
- Sertakan Langkah kerja program secara ruhut dan sistematis.

DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data Pekan 3

[2] Data Structures and Algorithms in Java